Ders 4 Genişletilmiş Notu: Quiz 1 Çözümleri, MATLAB ve Python ile Sinyal Çizimi, Sistemlere Giriş ve Sistem Özellikleri

Ders 4 Genişletilmiş Notu: Quiz 1 Çözümleri, MATLAB ve Python ile Sinyal Çizimi, Sistemlere Giriş ve Sistem Özellikleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Quiz 1 Sorularının Çözümü
- 2. Sinyal İşlemlerinin Birleştirilmesi: $x(-2t+6)$
- 3. Ayrık Zamanda Aynı İşlemler: $x[-n+2]$
- 4. Dürtü Bir Örnekleme Operatörüdür
- 5. Sinyallerin Toplanması: Bağımsız Bileşen Analizi
- 6. Sinyallerin Çarpılması: Slayt Örnekleri
- 7. Bilgisayardaki Sinyal Ayrık mı, Sürekli mi?
- 8. MATLAB Çizim Komutları
- 9. MATLAB ile Sönümlü Sinüzoid
- 10. Python ile Sinyal Çizimi
- 11. Karmaşık Sinyal: $e^{j100\pi t}$
- 12. Sonuçları Grafikle Raporlamak
- 13. Yedek Kaynaktan Örnekler ve Meksika Şapkası
- 14. Sistem Kavramı ve Blok Diyagramları
- 15. Temel Sistemler
- 16. Sistem Bağlantıları
- 17. İnsan Öğrenmesi Modeli ve PID Denetleyici
- 18. Doğrusallık
- 19. Doğrusal ve Doğrusal Olmayan Sistem Örnekleri
- 20. Bellekli ve Belleksiz Sistemler
- 21. Nedensellik
- 22. Zamanla Değişmezlik
- 23. Kararlılık
- 24. Örnek Sistem: Hız Sabitleyici
- 25. Terslenebilirlik
- 26. Diferansiyel Denklemlerle Tanımlanan Sistemler
- 27. Fark Denklemleriyle Tanımlanan Ayrık Sistemler

1. Quiz 1 Sorularının Çözümü

Hoca derse birinci quizin sorularını hızlıca çözerek başladı. Bu sorular örnekleme ve nicemleme kavramlarını hesap yaparak oturtmak için seçilmişti. Çözümler aşağıda sorular sırasıyla verilmiştir.

1.1 CD Kalitesinde Müzik

Soru: 4 dakikalık bir müzik CD kalitesinde, yani 44100 Hz örnekleme frekansıyla saklanacaktır. Sağlıklı kayıt için müziğin içindeki bileşenlerin frekansı en fazla kaç Hz olabilir?

Çözüm: Örnekleme frekansı, sinyaldeki en yüksek frekansın en az iki katı olmalıdır. O hâlde en yüksek frekans örnekleme frekansının yarısıdır:

$$f_{\max} = \frac{f_s}{2} = \frac{44100}{2} = 22050 \text{ Hz}$$

1.2 Analog Sinyallerle İlgili Yanlış İfade

Hoca bu soruyu biraz zor bulduğunu, bazı öğrencilerin burada yanılgıya düştüğünü söyledi. Şıklar tek tek şöyle değerlendirildi:

- **Bir veya daha fazla bağımsız değişkenli olabilme:** Doğru.
- **Dönüştürme kaybı:** Analog sinyalin dijitale çevrilmesinde oluşan kayıp, örnekleme sayısı ve nicemleme seviyesiyle ters orantılıdır. Doğru. Örnekleme ve nicemleme sayısı arttıkça kayıp azalır.
- **Hem analog hem dijital sistemlerde işlenebilme:** Doğru. Analog sistem varsa analog olarak işleriz. Dijital sistemde işleyeceksek dijitale çevirириz.
- **Teoride ve uygulamada analog sinyal, sonsuz kez örneklenmiş sinyal olarak tanımlanabilir:** Yanlış. Bu ifade teoride doğrudur. Bir sinyali sonsuz sıklıkta örneklersek teorik olarak analog sinyali elde ederiz. Ama uygulamada sonsuz kez örneklemek mümkün değildir. Sorunun cevabı bu şıktır.
- **Örnekleme frekansı, en yüksek frekansın iki katından fazla olmalıdır:** Doğru. Bu, baştan beri anlatılan genel kuraldır.

Soru: Analog sinyali dönüştürüp kullanırsak bu hibrit sistem olmaz mı? Slaytta analog, dijital ve hibrit sistemler ayrı sayılmıştı ve hibrit sistem, ADC ve DAC ile çevirip işleme olarak anlatılmıştı.

Cevap: Olabilir, ama şık hibrit sistemden söz etmiyor. Burada söylenen, her iki tarafta da işlemenin mümkün olduğudur. Mümkün olup olmamaktan (possibility) bahsediliyor. Örneğin insan sesini analog olarak işleyen sistemler de, dijital olarak işleyen sistemler de vardır. "Analog sinyal dijital olarak işlenemez" diye bir kural olsaydı şık yanlış olurdu.

Tartışma sorunun sonunda yeniden açıldı. Öğrenci şıkta "dijital sistem" dendiğini, hibrit sistemin ise ayrı bir kavram olarak anlatıldığını hatırlattı. Hoca, hibrit sistemin analoğu da dijitali de içeren bir kavram olduğunu, bunların birbirinden bağımsız olmadığını söyledi. Şıkta "hibrit sistemle gerçeklenmez" denseydi öğrencinin haklı olabileceğini ekledi. Konuyu bilen birinin zaten "teoride ve uygulamada" ifadesinin yanlış olduğunu doğrudan görmesi gerektiğini belirtti.

1.3 Görüntünün Kapladığı Yer

Soru: 256×256 çözünürlüklü, 256 renkli bir görüntü ne kadar yer kaplar?

Hoca önce örnek sayısını, sonra örnek başına bit sayısını buldu. Bu tarz soruların, örnekleme (sampling) ve nicemleme (quantization) kavramlarını hesap yaparak anlamak için sorulduğunu söyledi.

- **Örnekleme:** İki boyutlu örnek sayısı, yani piksel sayısı $256 \times 256 = 2^8 \times 2^8 = 2^{16}$'dır.
- **Nicemleme:** Her örnekte 256 renk varsa seviye sayısı (number of levels) 256'dır. Bu da $\log_2 256 = 8$ bit, yani 1 bayt eder.
- **Toplam:** $2^{16} \times 8$ bit $= 2^{16}$ bayt.
- **Birim dönüşümü:** $2^{16} = 2^6 \times 2^{10}$. Burada $2^6 = 64$, 2^{10} ise "kilo"dur. Sonuç 64 KB'dir.

Sonuç Python ile doğrulandı: $65536/1024 = 64$.

1.4 Derste Eklenen Soru: 10 Dakikalık Stereo Müzik

Hoca quizde olmayan bir soru daha sordu: Bir müzik 10 dakika boyunca CD kalitesinde örnekleniyor, ses kartının çözünürlüğü 16 bit ve kayıt stereo. Ne kadar veri oluşur?

- **Saniyedeki örnek:** 44100.
- **10 dakikadaki örnek:** $44100 \times 60 \times 10 = 26\,460\,000$.
- **16 bit nicemleme:** Örnek sayısı 16 ile çarpılır.
- **Stereo:** Sonuç 2 ile çarpılır.

Stereo, kaydın iki kanallı olması demektir. Hoca bunu iki mikrofon, iki kaydedici varmış gibi düşünmemizi söyledi. Stereo bir ses dosyasını açınca içinde birbirine çok yakın iki dizi görürüz. Aradaki ufak farklar mikrofonların konumu gibi şeylerden gelir.

Hoca sayıları düzgün olmadığı için sonucu derste hesaplamadı. Bulunan değerin 1024'e bölünerek daha büyük birimlere çevrileceğini söylemekle yetindi. Sonuç şöyledir (ara adım eklendi, Python ile hesaplandı):

$$44100 \times 600 \times 16 \times 2 = 846\,720\,000 \text{ bit} = 105\,840\,000 \text{ bayt} \approx 103\,359{,}4 \text{ KB} \approx 100{,}9 \text{ MB}$$

Burada KB ve MB, 1024 tabanıyla hesaplanmıştır.

1.5 Sinüzoidal İşaret

Soru: $x(t) = 100\sin(500\pi t + 90^\circ)$ işaretiyle ilgili hangi ifade yanlıştır?

- **Genlik 100, faz 90°:** Doğru.
- **Frekans:** Sinüzoidal işaret $\sin(2\pi f t)$ biçiminde yazılır. $2\pi f = 500\pi$ olduğundan $f = 250$ Hz'dir.
- **Örnekleme frekansı en az 500 Hz olmalıdır:** Doğru. 250 Hz'in iki katıdır.
- **1000 Hz ile örnekleme:** Doğru. Frekansın 4 katıyla örneklemekte bir engel yoktur.
- **İşarete $2t$ dönüşümü yapılırsa işaret genlik ekseninde sıkıştırılır:** Yanlış. Sorunun cevabı budur. Bu işlem genlik ekseninde yapılmaz, zaman yani yatay eksende yapılır. İşaret zaman ekseninde sıkıştırılır.
- **İşaret periyodiktir:** Doğru. Sinüzoidler doğası gereği periyodiktir.

Soru: 250 değerini nereden bulduk?

Cevap: Sinüzoidal işaret $2\pi f t$ ile gösteriliyordu. Bunu $500\pi t$ ile eşleştirirsek $f = 250$ çıkar.

1000 Hz şıkkında t yerine nT_s veya n/f_s yazılır:

$$x[n] = 100\sin\left(500\pi\frac{n}{1000} + 90^\circ\right) = 100\sin\left(\frac{\pi n}{2} + 90^\circ\right)$$

Python ile hesaplandı. İlk değerler $100, 0, -100, 0, \ldots$ biçimindedir ve dizi $100\cos(\pi n/2)$ ile aynıdır.

> **Ders sonrası not:** Derste aynı kural bir yerde "iki katından fazla", burada "en az iki katı" diye geçti. Kesin ifade "iki katından fazla"dır. Tam iki katta örnekleme, sinüzoidin fazına göre bilgi kaybına yol açabilir.

1.6 Tek mi, Çift mi?

Soru: $x(t) = t^2 + 2$ sinyali tek midir, çift midir?

Hoca bu sorunun kuralı işletince kolayca çözüldüğünü söyledi. Bir sinyal $x(t) = x(-t)$ ise çift, $x(-t) = -x(t)$ ise tektir.

$$x(-t) = (-t)^2 + 2 = t^2 + 2 = x(t)$$

Birinci koşul sağlanır, ikincisi sağlanmaz. Sonuç: sinyal çifttir, tek değildir.

1.7 Nicemleme: Seviye Sayısı ve Doğruluk

Soru: Bir ses kayıt cihazı -20 V ile $+20$ V arasında değer alan sinyalleri nicemleyecek ve örnek başına 16 bit kullanacaktır. Genlik ekseninin bölüneceği bölge sayısı L ve sayısallaştırmada oluşan doğruluk değeri Δ kaçtır?

Hoca örnek olarak bir sinüzoid düşünmemizi, ama sinyalin sürekli olmak zorunda olmadığını söyledi. Önemli olan üst nokta ile alt nokta arasındaki aralıktır.

- **Aralık:** $20 - (-20) = 40$ V.
- **Seviye sayısı:** $L = 2^{16} = 65536$.
- **Doğruluk (accuracy):** $\Delta = \dfrac{40}{2^{16}} \approx 0{,}00061$ V.

  Ders sonrası not: Hoca değeri sözlü olarak "0,006 herhalde" diye söyledi. Python ile hesaplandı: $40/65536 = 0{,}0006103515625$ V. Yani doğru değer yaklaşık $0{,}00061$ V'tur, söylenenden 10 kat küçüktür.

Soru: Burada doğruluk değeri değil de nicemleme hatası (quantization error) sorulması gerekmiyor mu?

Cevap: Slaytta doğruluk değeri de yazıyor. Nicemleme hatası veya doğruluk (accuracy), ikisi de aynı kavramlardır.

> **Ders sonrası not:** Δ iki komşu seviye arasındaki adımdır. Değer en yakın seviyeye yuvarlanırsa en büyük nicemleme hatası bunun yarısıdır: $\Delta/2 \approx 0{,}00031$ V (Python ile hesaplandı). Bu yüzden "doğruluk" adım olarak verilmişse, en büyük yuvarlama hatasıyla sayısal olarak aynı değildir.

2. Sinyal İşlemlerinin Birleştirilmesi: $x(-2t + 6)$

Hoca bu konuda çok hata yapıldığını, öğrencilerin de sorduğunu söyleyerek konuya döndü. Üç temel işlemimiz vardır: sinyali tersleme (reverse), öteleme (shift) ve ölçekleme (scale).

> **Hocanın vurgusu:** Bu işlemlerin hepsi t veya n ekseninde, yani yatay eksende yapılır. Genlik ekseninde yapılmaz.

Karşımıza $x(-3t + 5)$ gibi bir ifade geldiğinde ne yapacağız? İçinde ölçekleme, öteleme ve tersleme vardır. Yöntemlerin tanımında genelde atlanan nokta şudur: işlem t üzerinde yapılır. $x(t)$'ye tersleme uygularsam $x(-t)$ olur. Artık elimdeki yeni sinyal $x(-t)$'dir. Bundan sonraki işlemler bu sinyal üzerinde, yine t'nin yerine yazılarak yapılır.

Karışıklık genelde ötelemede çıkar. Matematikteki işlem önceliği ve ters fonksiyonlar çağrışım yapar. Hoca bu üç işlemin komütatif olduğunu, herhangi bir işlem önceliğinin olmadığını söyledi ve bunu aynı hedefe üç farklı yoldan giderek gösterdi. Hedef, $x(t)$'den $x(-2t + 6)$'yı elde etmektir.

> **Ders sonrası not:** Hoca "üç işlem de komütatiftir" dedi. Daha doğrusu: işlemler istenen sırada yapılabilir ve aynı sonuca ulaşılır, ama öteleme miktarı seçilen sıraya göre değişir. Aynı parametrelerle sıra değiştirmek aynı sonucu vermez. Bir test sinyalinde Python ile denendi: önce 6 birim öteleyip sonra sıkıştırmak $x(2t + 6)$ verir. Önce sıkıştırıp sonra 6 birim ötelemek $x(2t+12)$ verir. Yalnız tersleme ile ölçekleme kendi aralarında yer değiştirebilir.

2.1 Birinci Yol: Önce Öteleme

1. **Öne alma (advance), 6 birim:** Hoca öne almayı "geriye alma" diye de adlandırdı. Grafikte sinyal sola gider. $x(t) \to x(t + 6)$.
2. **Tersleme:** Bundan sonraki işlem yine t üzerindedir, 6 üzerinde değildir. Tersleme parametre almaz. $t \to -t$ yazılır: $x(-t + 6)$.
3. **2 ile sıkıştırma:** $t \to 2t$ yazılır: $x(-2t + 6)$.

Hoca burada birden büyük bir sayı ile çarpmanın ne demek olduğunu da hatırlattı. $1/2$ ile çarpmak germe (stretch), 2 ile çarpmak sıkıştırma (compress) demektir.

Tahtada herhangi bir dalga formu üzerinde adımları gösterdi. 6 birim öne alınan sinyalin uçları -5 ve 2 olsun. Terslenince uçlar -2 ve 5 olur. 2 ile sıkıştırınca uçlar -1 ve $2{,}5$ olur (son değerler ara adım olarak eklendi, Python ile hesaplandı).

2.2 İkinci Yol: Önce Sıkıştırma

1. **2 ile sıkıştırma:** $x(t) \to x(2t)$.
2. **Öne alma:** Burada dikkat edilmesi gereken yer gelir. $x(2t)$'ye 6 birim öne alma uygulayamayız, çünkü işlem t üzerindedir. Başındaki 2 bizi engeller. $x(2t+6)$'yı elde etmek için 3 birim öne alırız:

$$x\big(2(t+3)\big) = x(2t+6)$$

3. **Tersleme:** $t \to -t$ yazılır: $x(2(-t)+6) = x(-2t+6)$.

2.3 Üçüncü Yol: Önce Tersleme

1. **Tersleme:** $x(t) \to x(-t)$.
2. **Öteleme:** Şimdi $-t+6$ bulmak istiyoruz. Başta eksi olduğu için buradaki t'ye tersine bir işlem, yani 6 birim gecikme (delay) uygularız:

$$x\big(-(t-6)\big) = x(-t+6)$$

3. **2 ile sıkıştırma:** $t \to 2t$ yazılır: $x(-2t+6)$.

Üç işlem olduğu için aslında altı farklı sıra vardır. Bu kuralla hepsi aynı sonucu verir. Hoca üçünü çözdü, kalan sıralar da eklenerek gereken öteleme aşağıda verilmiştir (ara adım eklendi, Python ile hesaplandı):

Sıra	t üzerindeki öteleme	Ara ifade
Öteleme, tersleme, sıkıştırma	$t \to t+6$	$x(t+6) \to x(-t+6) \to x(-2t+6)$
Öteleme, sıkıştırma, tersleme	$t \to t+6$	$x(t+6) \to x(2t+6) \to x(-2t+6)$
Sıkıştırma, öteleme, tersleme	$t \to t+3$	$x(2t) \to x(2t+6) \to x(-2t+6)$
Sıkıştırma, tersleme, öteleme	$t \to t-3$	$x(2t) \to x(-2t) \to x(-2t+6)$
Tersleme, öteleme, sıkıştırma	$t \to t-6$	$x(-t) \to x(-t+6) \to x(-2t+6)$
Tersleme, sıkıştırma, öteleme	$t \to t-3$	$x(-t) \to x(-2t) \to x(-2t+6)$

2.4 Kolay Sıra

İşi kolaylaştırmak için kaynaklarda genelde şu sıra önerilir: önce öteleme, sonra ölçekleme ve tersleme. Son ikisinin sırası değişebilir. Yine de hoca kısa yoldan çok işin kendisini öğrenmeyi önemsediğini söyledi. Hedef işarete giderken yapılan gecikme, öne alma, ölçekleme veya tersleme hep t üzerindedir.

3. Ayrık Zamanda Aynı İşlemler: $x[-n+2]$

Sürekli zamanda yapılan bu işlemler n düzleminde de aynı şekilde yapılır. Hoca bir öğrencinin sorduğu slayt örneğini açtı: Bir $x[n]$ dizisinden $x[-n+2]$ istenmektedir.

Önce terslemek istersek, $-n$ hâline getirdikten sonra artı iki birim uygulayamayız. Artı ikiyi dönüştürüp öyle uygulamak gerekir. Kısa yol, önce ötelemeyi yapmaktır:

1. **2 birim öne alma:** $x[n+2]$. Örnekler 2 birim sola gelir. $n=4$'teki örnek $n=2$'ye, $n=3$'teki örnek $n=1$'e gelir.
2. **Tersleme:** $x[-n+2]$. Yeni dizide $n=2$'deki örnek $n=-2$'ye, $n=1$'deki $n=-1$'e, $n=-1$'deki $n=1$'e gider. $n=0$'daki örnek yerinde kalır.

Soru: a şıkkındaki gösterimi gösterebilir misiniz?

Cevap: O şık $x[n-2]$'dir. Burada bize $n-2$ değil, önce $n+2$ lazımdır.

4. Dürtü Bir Örnekleme Operatörüdür

Hoca örneğe gelmişken bazı özel tanımları da söyledi ve $\delta(t-1{,}5)$ işaretini sordu. Bu işaret yalnız $t=1{,}5$ anında değer verir, onun dışında sıfırdır. Çünkü dürtünün argümanı sıfırken sonuç vardır, diğer durumlarda sonuç sıfırdır.

> **Ders sonrası not:** Hoca değeri "bir veriyordu" dedi. Bu, ayrık $\delta[n]$ için doğrudur. Sürekli $\delta(t)$'nin $t=0$'daki değeri 1 değildir. Alanı 1'dir. Çarpımdaki özelliği $x(t)\delta(t-t_0)=x(t_0)\delta(t-t_0)$ biçimindedir.

Peki bu işaret sürekli sinyallerde hangi operatör olarak kullanılabilir? Hoca $t-1{,}5$ yerine $t-k$ yazdı ve k'yı $-\infty$'dan $+\infty$'a değiştirdi:

$$\sum_{k=-\infty}^{\infty} \delta(t-k)$$

Soru: Bu ne için kullanılır? Bir öğrenci "saat" diye cevap verdi.

Cevap: Saat olmaz. Saat olması için u kullanmak gerekir, çünkü saat işareti belli bir süre aktif kalır, sonra düşer. Burada her durumda bir örnek çıkar.

Bu, aslında bir **örnekleme operatörüdür**. Örnek alacağımız k anında bir işaretle çarptığımızda o andaki değeri bırakır, diğer anları sıfırlar. k değerlerini toplam sembolüyle ayarlarız. Örneğin k, -5'ten $+5$'e gidebilir. Birer birer değil başka aralıklarla da değişebilir. Böylece istediğimiz aralıkta örnek alırız.

> **Hocanın vurgusu:** Bu örnekleri çözmeye çalışın. Sınavda bu tarz bir soru geliyor: Bir sürekli veya ayrık zaman sinyali verilip buradan şu sonucu bulun denebilir.

5. Sinyallerin Toplanması: Bağımsız Bileşen Analizi

Sinyaller birbiriyle toplanabilir ve çarpılabilir. Hoca çarpma örneğinden önce toplamaya ilginç bir uygulama gösterdi.

Önce problem: 5, 10 ya da genel olarak n kişi bir yerde konuşuyor. Sesler n mikrofonla kaydediliyor. Kaydedilen sinyallerden kimin ne söylediğini ayırt etmek istiyoruz. Buna **kokteyl partisi problemi** (cocktail party problem) denir. Onu çözen birçok algoritma vardır. Bunlardan biri **bağımsız bileşen analizidir** (independent component analysis, ICA). Probleme kör kaynak ayrıştırma (blind source separation) adı da verilir.

Yapılan şey şudur: benim konuşma sinyalim ile başkasının sinyali toplanır. İstenen, bu toplamı geri açmak, kaynakları yeniden ortaya çıkarmaktır.

Hoca önce basit sinyallerle yapılmış bir örnek gösterdi. Bir sinüzoidal sinyal ve bir kare dalga üretilmiş, birine gürültü eklenmiş. Sonra iki veri karıştırılmış. Hoca karıştırmayı burada bir toplama gibi düşünmemizi söyledi. Sonuçta yeni bir dalga formu çıkar. ICA bu dalga formundan yola çıkarak kaynakları elde etmeye çalışır. Sonuç birebir değildir: Grafikteki yeşil bileşende belli bir miktar öteleme, faz değişimi olmuştur.

İkinci örnek konuşma sesleriydi. İki ayrı kaynağın iki farklı karışımı (iki gözlem) dinletildi. Yöntem bu iki gözlemden yola çıkarak önce birini, sonra diğerini buldu. Böylece kaynaklar ayrıştırıldı (decomposition).

Ek bilgi (derste anlatılmadı): Karışım büyük olasılıkla bir karışım matrisiyle yazılan ağırlıklı toplamdır: $\mathbf{x} = A\mathbf{s}$. Burada $\mathbf{s}$ kaynakları, $\mathbf{x}$ gözlemleri gösterir. ICA büyük olasılıkla A'yı bilmeden, kaynakların istatistiksel olarak bağımsız olduğu varsayımıyla bir ayırma matrisi arar. Kişi kadar mikrofon istenmesinin sebebi de büyük olasılıkla bu matrisin kare ve çözülebilir olmasıdır.

6. Sinyallerin Çarpılması: Slayt Örnekleri

Hoca toplamadan sonra slayttaki çarpma örneklerine geçti. Verilen sinyal şöyledir: $-1 \leq t \leq 0$ arasında 0'dan 1'e doğrusal artar, $0 \leq t \leq 1$ arasında 1'dir, $1 \leq t \leq 2$ arasında 2'dir, diğer yerlerde 0'dır.

6.1 $x(t)u(1-t)$

Bu madde kısa notta slayt örneği olarak geçiyor. Altyazıda ayrıca çözülmüyor. $u(1-t)$, $t \leq 1$ için 1'dir. Çarpımda sinyalin $t > 1$ kısmı silinir, geri kalanı aynı kalır.

6.2 $x(t)[u(t)-u(t-1)]$

Hoca önce çarpılan ifadeyi tanımladı. Bu da t üzerinde yapılan bir işlemdir. $u(t)$, sıfırdan sonsuza kadar giden bir basamaktır. Bundan $u(t-1)$ çıkarınca 1'den sonrası tamamen gider. Elimizde 0 ile 1 arasında bir dikdörtgen kalır.

Bir sinyali bununla çarpmak bir **filtre** gibidir. Sıfır ile bir arasındakileri alır, kalanları bastırır. Çünkü dikdörtgenin dışında sıfırlar vardır ve bu sıfırlar sinyalin kalanını yok eder. Sonuç: sinyalin yalnız $0 \leq t < 1$ kısmı kalır. Bu kısımda değer 1'dir. Hoca kendi çözümünü slayttaki sonuçla karşılaştırdı, aynı çıktı. Python ile de hesaplandı.

6.3 $x(t/2)\delta(t-1{,}5)$

Burada önce $x(t/2)$ bulunur, sonra yeni sinyalin $t = 1{,}5$'teki değeri alınır.

- $x(t/2)$**:** Hoca önce "sıkıştırıyoruz" dedi, sonra kendini düzeltti. Sinyal 2 kat gerilir. $x(t)$'nin 1'deki değeri 2'ye, 2'deki değeri 4'e, -1'deki değeri -2'ye gider. Genlik değişmez.
- **Yeni sinyalin değerleri:** 0 ile 2 arası artık 1 değerindedir, 2 ile 4 arası 2 değerindedir.
- **Dürtüyle çarpım:** $t = 1{,}5$, 0 ile 2 aralığına düşer. Yalnız bu andaki değer kalır, geri kalan her şey bastırılır:

$$x(t/2)\,\delta(t-1{,}5) = x(0{,}75)\,\delta(t-1{,}5) = 1 \cdot \delta(t-1{,}5)$$

Hoca slayttaki cevapta dürtünün ağırlığının 2 gösterildiğini fark etti. Slaytın bu değeri $x(t)$ üzerinden, yani $x(1{,}5) = 2$ ile bulduğunu, $x(t/2)$ için doğrusunun 1 olduğunu söyledi. Slayttaki önceki çözümde hata olduğunu belirtti. Python ile dar bir Gauss dürtüsüyle integral alındı, sonuç 1 çıktı.

6.4 En Sık Yapılan Hata

Hocanın vurgusu: İşlemin t ekseninde, ayrık zamanda ise n ekseninde yapıldığına ve hedefe giderken hangi işlemin nasıl yapılacağına dikkat edin. Az örnek çözdüyseniz bu konu can yakabilir.

Genelde şu yapılıyor: $x(t)$'den $x(2t)$ elde ediliyor, sonra "buna 6 birim eklerim" deniyor. 6 birim eklemek $x(2(t+6)) = x(2t+12)$ yapmak demektir. Doğrusu t'yi 3 birim ötelemektir, böylece $x(2t+6)$ bulunur.

Lisedeki fonksiyon konusu buna benzer, ama aynı şey değildir. $y = f(x) = x + 5$ dediğimizde y, x'e göre değişir. Bizim burada $y(t) = x(t+5)$ dediğimiz şeyde ise y, x'e göre değişirken x de t'ye göre değişir. Girişte de bir değişim vardır. Hoca fonksiyonlardaki çağrışımla yanlış yapmamamız gerektiğini söyledi.

7. Bilgisayardaki Sinyal Ayrık mı, Sürekli mi?

Hoca derste normalde çok ayrıntılı işlenmeyen sinyal çizimini anlatmak istediğini söyledi. Deney yapar gibi, $x(t)$'den $y(t)$, $y(t)$'den $z(t)$ elde ettiğimizde sonuçları görselleştirebilmemiz gerekir. MATLAB veya Python bu yüzden tercih edilir, C'de kod yazmak istenmez. Scilab veya Octave gibi araçlar da kullanılabilir.

Elimizde bir $x(t)$ olsun. Üzerinde işlem yapıp $y(t)$, sonra $z(t)$ bulalım. Ya da iki boyutlu düşünelim, bir görüntüye işlem uygulayalım. Bütün değişimleri görebilmek için hepsi tek pencerede çizilir. Grafiğin üzerine çizildiği bu alana **tuval** (canvas) denir.

Hoca MATLAB'de üç değişken oluşturdu:

```
t = 1:10;
x = 10*cos(2*pi*50*t);
y = 10*sin(2*pi*50*t);
```

Sonra sordu: t, x ve y ayrık mı, sürekli mi? Hoca bunun çok yapılan hatalardan biri olduğunu söyledi.

Soru (hoca): Bu değişkenler ayrık mı, sürekli mi?

Cevaplar: Bir öğrenci "t'den dolayı ayrık" dedi, t'yi kağıtta sürekli gösterdiğimizi ama burada tek tek örnek aldığımızı ekledi. Bir diğeri x ve y'nin sürekli, t'nin parçalı olduğunu öne sürdü. Bir başkası da "sinüs ve kosinüs fonksiyonları her zaman sürekli değil mi?" diye sordu.

Cevap (hoca): "t'den dolayı ayrık" demek tam yerinde değildir, çünkü aralığı ve örnek sayısını değiştirebiliriz. Sinüs ve kosinüsün ayrık hâlleri de vardır. Onlar için söyleyebileceğimiz şey periyodik olduklarıdır. Asıl soru şudur: Bu sinyalleri nerede saklıyoruz?

> **Hocanın vurgusu:** Bilgisayarda saklıyorsam her türlü ayrıktır. Elle üretilmiş sanal bir işaret bile olsa, bilgisayardaki her işaret ayrıktır.

Eşitliği kağıt üzerinde düşününce sürekli gibi görünür. Ama çizdirdiğimizde gördüğümüz süreklilik, çizim komutunun yaptığı bir şeydir.

7.1 `plot` ve `stem`

Hoca `plot` komutunda önce yatay ekseni, sonra dikey ekseni yazdığımızı hatırlattı: `plot(t, x)`. İlk denemede grafik beklenen gibi görünmedi. Hoca bunun t ile ilgili olabileceğini söyleyip t'yi `t = 1:0.01:100` yaptı.

> **Ders sonrası not:** İlk denemedeki görüntünün sebebi hesapla bulundu. $t = 1, 2, \ldots, 10$ tam sayıdır, bu yüzden $\cos(2\pi \cdot 50\, t) = 1$ ve $\sin(2\pi \cdot 50\, t) = 0$ olur. x sabit 10, y sıfır çıkar. `t = 1:0.01:100` ile örnekleme frekansı $f_s = 100$ Hz olur, bu da 50 Hz'in tam iki katıdır. Bu kez x dizisi $+10, -10, +10, \ldots$ diye gider, y ise yuvarlama hatası dışında sıfırdır (Python ile hesaplandı, $|y| < 10^{-10}$). Bu, Bölüm 1.5'teki notun somut hâlidir.

- **`plot`:** Herhangi iki örnek arasındaki aralığı doldurur. Süreklilik varmış gibi gösterir.
- **`stem`:** Ayrık zamanlı sinyal çizer. Çok değer varsa karman çorman görünür, bu normaldir. Yakınlaştırınca ayrık değerler görülür.

Hoca bu iki farklı fonksiyonun birçok dilde bulunduğunu söyledi.

7.2 `hold on`

Çizilen her grafik bütün dillerde bir şekil (figure) nesnesine aktarılır. Yeni bir grafiği aynı şeklin üzerine çizmek için MATLAB'de `hold on` kullanılır. Üst üste çizilince karışmaması için renk verilir:

```
plot(t, x);
hold on
plot(t, y, 'r');
```

8. MATLAB Çizim Komutları

8.1 `figure`

Aradan sonra hoca yeni bir t oluşturdu (`t = 1:0.1:10`) ve bir z sinyali çizdi. Ardından tekrar `plot(x)` dedi. Değişken x hâlâ çalışma alanında (workspace) durur. `hold on` yoksa yeni grafik eskisini siler. Ayrı ayrı görmek için yeni çizimden önce `figure` yazılır. Yeni bir pencere açılır, biri bir pencereye, diğeri öbürüne çizilir. Hoca bunun MATLAB'in masaüstü sürümünde daha kolay olduğunu belirtti.

8.2 `subplot`

Bütün değişimi aynı anda görmek istiyorsak `subplot` kullanılır. Bir ekranda, tek bir tuval üzerinde birden fazla grafik gösterir. Hoca `help subplot` ile belgeleri açtı. Belgedeki örneklerle şunları gösterdi:

- `subplot(2,1,1)`: İki satır, bir sütunluk alan açar ve birinci bölüme çizer. `subplot(2,1,2)` ikinci bölüme çizer.
- `subplot(2,2,k)`: Dört grafik için kullanılır. k değerleri ID gibi düşünülür. Belgedeki örnekte sırayla $\sin x$, $\sin 2x$, $\sin 4x$ gibi grafikler çizilir.
- **Farklı boyutlar:** İki satır, iki sütunluk alanda 1. ve 2. grafikler ayrı çizilip 3. ve 4. yerler birleştirilebilir.

Aynı yapı `stem` ile de kullanılabilir. Belgedeki bir örnekte `for` döngüsüyle rastgele veri üretilip dört bölüme `stem` ile çizilmişti. `subplot` altında çubuk grafik (bar) veya kutupsal grafik (polar plot) gibi başka çeşitler de çizilebilir.

8.3 Temizlik Komutları

`clear` çalışma alanındaki bütün değişkenleri siler. `clc` komut penceresini temizler.

9. MATLAB ile Sönümlü Sinüzoid

Hoca hem MATLAB'de hem Python'da bir sinyal çizmeye geçti. MATLAB örneği devre teorisinde de görülen sönümlü bir dalgaydı. Sinyal iki temel sinyalin çarpımıdır: bir sinüs ve bir üstel.

9.1 Sinyalin Davranışı

Kod yazmadan önce hoca sinyalin nasıl davranacağını sordu. Sinüs -1 ile 1 arasında değişir. Üstel kısım bu sinüsün önündeki katsayı gibi düşünülebilir, ama bu katsayı t'ye göre değişir.

- **Başlangıç değeri (initial value):** $e^0 = 1$ olduğundan üstel kısım önündeki katsayıdan başlar.
- **Limit değeri (limit value):** Üs $-\infty$'a giderken $e^{-\infty} = 1/e^{\infty} = 1/\infty \to 0$ olur.

Bu yüzden dalga her salınımda geldiği yere dönmez, biraz daha düşükten gider, sıfıra doğru sönümlenir. Hoca başlangıç ve limit değer problemlerini ileride göreceğimizi söyledi.

9.2 Kod

MATLAB'de bağımsız değişkeni oluşturmak çok kolaydır. Hazır fonksiyonlar için ek bir şey yüklemek gerekmez. Komutlar önce tek tek yazıldı, sonra aşağıdaki gibi bir araya getirildi:

```
t  = 0:0.01:250;
x1 = exp(-0.02*t);        % sönen üstel
x2 = sin(10*pi*t/250);    % sinüzoid
x  = x1 .* x2;            % eleman eleman çarpım

subplot(3,1,1); plot(t, x1, 'r'); axis auto; grid on;
xlabel('Time (s)'); ylabel('Amplitude');
legend('Exponential Signal');
```

```
title('Signal Plotting Using MATLAB');

subplot(3,1,2); plot(t, x2, 'b'); axis auto; grid on;
xlabel('Time (s)'); ylabel('Amplitude');
title('Sinusoidal Signal');

subplot(3,1,3); plot(t, x, 'g'); axis auto; grid on;
xlabel('Time (s)'); ylabel('Amplitude');
```

Bu kod MATLAB'de çalıştırılmadı. Aynı sinyaller Python'a çevrilip hesaplandı ve çizdirildi.

Kodla ilgili noktalar şunlardır:

- **`.*` işlemi:** x_1 ve x_2 vektördür. Hoca önce `x1*x2` yazdı, sonra `.*` olması gerektiğini söyledi.
- **`axis auto`:** Eksen aralıkları otomatik ayarlanır. Çünkü bir sinyal 0-1 arasında, diğeri başka bir aralıkta değişebilir.
- **`grid on`:** Arka planda ızgara çizgileri gösterir.
- **`xlabel, ylabel`:** Hoca akademik çalışmalarda eksen etiketinin çok kullanıldığını söyledi. Y ekseni, volt veya amper gibi özel bir çıkış yoksa genel olarak genliktir (amplitude).
- **`legend`:** Haritalardaki lejant gibi, sinyalin ne olduğunu gösterir.
- **`title`:** Grafiğin başlığını verir.

Üç grafikte beklenen görüldü. Birinci grafikteki üstel 1'den başlıyor ve sıfıra iniyor. Üçüncü grafikteki sinyal sönümlenerek sıfıra ulaşıyor.

> **Ders sonrası not:** Sayısal değerler Python ile hesaplandı. Örnek sayısı 25001'dir. $x_1(250) = e^{-5} \approx 0{,}0067$'dir. x_2'nin periyodu $2\pi/(10\pi/250) = 50$ saniyedir, yani 250 saniyede 5 periyot vardır. Hoca davranışı anlatırken "kosinüs gibi başlatalım" diyerek dalganın katsayıdan başladığını söyledi. Koddaki x_2 sinüs olduğu için $x(0) = 0$'dır, başlangıç genliği zarfta görülür.

9.3 Betik ve Dışa Aktarma

Komutları tek tek yazmak yerine `.m` uzantılı bir dosyaya, yani bir betiğe (script) yazıp tek seferde çalıştırmak da mümkündür. Hoca bunu "New Script" ile yaptı, `myplot.m` olarak kaydetti ve komut penceresinden `myplot` yazarak çağırdı. Adım adım yazmayı anlatırken daha iyi bulduğunu söyledi. Bir kez hazırlanan betikte yalnız gerekli yerleri değiştirmek yeter. Bunun için nesneye yönelik bir programlama bilgisi gerekmez.

Grafik `Export` ile PNG olarak dışa aktarılabilir ve rapora eklenebilir. MATLAB'in masaüstü sürümünde grafik üzerinde değişiklik de yapılabilir, üzerine tıklayarak değerler okunabilir. Hoca web arayüzünde bunun o kadar iyi olmadığını belirtti.

10. Python ile Sinyal Çizimi

Hoca Python'da işin biraz daha zor göründüğünü, ama aslında çok zor olmadığını söyledi. Fark, ek kütüphane gerekmesidir: `t`'yi oluşturmak için NumPy, çizim için matplotlib. Hoca Spyder ortamını kullandı. Spyder'da MATLAB'e benzer şekilde kod yazılır, değişkenler de "Variable Explorer" penceresinde görülür.

10.1 İlk Hatalar

Canlı denemede iki hata çıktı, ikisi de öğreticidir:

- **`np is not defined`:** NumPy içe aktarılmadan kullanılmıştır. `import numpy as np` yazıldıktan sonra her yerde `np` kullanılabilir.
- **`matplotlib` içinde `plot` yok:** Hoca `import matplotlib as plt` ile denedi. Doğrusu `pyplot` modülünü içe aktarmaktır: `import matplotlib.pyplot as plt`.

İkinci hatalı yazım denendi. Python `module 'matplotlib' has no attribute 'plot'` hatası veriyor.

10.2 Kod

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-0.02, 0.05, 1000)
plt.plot(t, 325*np.sin(2*np.pi*50*t))
plt.xlabel('t')
plt.ylabel('Amplitude')
plt.title('Signal Plotting Using Python NumPy and Matplotlib')
plt.xlim(-0.02, 0.05)
plt.show()
```

Kod matplotlib 3.11 ile çalıştırıldı, 50 Hz'lik sinüs çizildi. Burada π de NumPy'dan gelir: `np.pi`. Python'da hazır fonksiyonlar doğrudan gelmediği için yazım biraz daha ayrıntılıdır. `plt.show()` en sonda yazılır. Çıkan şekil "Save plot as" ile PNG olarak kaydedilebilir.

Çizilecek aralık, oluşturulan aralıktan ayrı bir şeydir. Otomatik çizimde eksen ilk ve son değere göre düzenlenir. Her yeri çizdirmek istemeyebiliriz. Bunu `plt.xlim` ile kendimiz belirleriz.

> **Ders sonrası not:** Hoca `np.linspace`'in üçüncü parametresini önce "artış miktarı" gibi anlattı, sonra "bine kadar git" dedi. Doğrusu: `np.linspace(baş, son, N)` iki uç arasında eşit aralıklı N örnek üretir (denendi). Burada adım $0{,}07/999 \approx 7 \times 10^{-5}$ s'dir.

Hoca logaritmik aralıklardan da söz etti. `linspace` eşit aralıklı artış üretir. Bazı durumlarda doğrusal ölçek doğru fikir vermez. Bu yüzden özellikle seste grafik logaritmik ölçekte çizilir. MATLAB'de bunun için hazır bir yol olduğunu söyledi.

> **Ders sonrası not:** Hoca Python'da aradı ve "doğrudan yok" dedi. NumPy'da `np.logspace` vardır (denendi).

10.3 Ayrık Gösterim ve Örnek Sayısı

Ayrık çizmek için `plot` yerine `stem` yazılır: `plt.stem(t, x)`. 1000 örnekte `stem` bile sürekli gibi görünür. Hoca bunun için örnek sayısını azalttı.

- **10 örnek:** Hoca örnek sayısının yeterli olmadığını söyledi.
- **100 örnek:** Sinüzoidal dalganın karakteristiğini çok iyi yansıtıyor.
- **75 ve 50 örnek:** Yine yansıtıyor.
- **40 örnek:** Hoca bunun örnekleme teoremine birazcık aykırı olduğunu söyledi. Sinüs dalgası kosinüse geçiyor, dalganın bir yerinde önceki kısımlara göre faz farkı görülüyor. Bunu örtüşme (overlapping) olarak ileride anlatacağını belirtti.
- **30 örnek:** Daha ciddi bir faz farkı görülüyor.

> **Ders sonrası not:** Sinyalin frekansı 50 Hz'dir (derste bir yerde 25 diye geçti). Yani $f_s > 100$ Hz yeterlidir. Hoca 30 ve 40 örneği teoreme aykırı saydı. Hesaba göre bu durumlar teoreme aykırı değildir ve örtüşme yoktur. 10 örnek bile bu aralıkta sınırın üstündedir. Spektrumun tepesi 30 ve 40 örnekte 50 Hz'de çıktı (Python ile hesaplandı):

Örnek sayısı	f_s (Hz)	Periyot başına örnek
10	128,6	2,57
30	414,3	8,29
40	557,1	11,14
50	700,0	14,00
75	1057,1	21,14
100	1414,3	28,29
1000	14271,4	285,43

> Görülen bozulmanın sebebi başkadır. Periyot başına örnek sayısı azdır ve tam sayı değildir. Bu yüzden örnekler her periyotta dalganın farklı yerlerine düşer ve her

periyot farklı görünür. 50 örnekte değer tam 14 olduğu için bütün periyotlar aynı görünür. Görüntü bir görselleştirme etkisidir, frekans bilgisi kaybolmamıştır. Aralık $-0{,}05$ ile $0{,}05$ alınsa da 30 ve 40 örnekte f_s sırayla 290 ve 390 Hz olur, sonuç değişmez. Yalnız 10 örnek bu aralıkta 90 Hz eder ve sınırın altına düşer.

11. Karmaşık Sinyal: $e^{j100\pi t}$

Hoca `subplot` için karmaşık değerli bir sinyal örneği seçti. Kodların ekran görüntüsünü alabileceğimizi de söyledi.

$$x(t) = e^{j100\pi t}$$

Soru (hoca): Bu işaret gerçel mi, sanal mı, karmaşık mı?

Cevap: Karmaşıktır. Açılımı Euler formülüyle yazılır:

$$e^{j100\pi t} = \cos(100\pi t) + j\sin(100\pi t)$$

> **Hocanın vurgusu:** Sinyal işleme konularında çalışacaksanız Euler formülünü ezbere bilmeniz gerekiyor.

Hoca $\mathrm{Re}\{x(t)\}$'nin ne olduğunu sordu. Bir öğrenci önce sinüslü kısmı söyledi. Hoca R'nin gerçel (real) demek olduğunu hatırlatınca cevap kosinüs kısmı oldu: $\mathrm{Re}\{x(t)\} = \cos(100\pi t)$. MATLAB'de de Python'da da `real` fonksiyonu vardır. Karmaşık bir sinyalin gerçel kısmını verir. `imag` ise sanal kısmı verir.

Gerçel ve sanal kısım arasındaki ilişki j dışında kosinüs ile sinüs arasındaki ilişkidir. Aralarında 90° fark vardır. Bu formüller toplam-fark formüllerinden çıkarılabilir. Hoca bunun için çok ciddi trigonometri bilgisi gerekmediğini, ama bu dönüşümleri bilmenin çok şey kazandırdığını söyledi.

> **Ders sonrası not:** Hoca eşitlikleri "± 90" ile yazdı. İşaret önemlidir. $\cos(\theta - 90^\circ) = \sin\theta$, $\cos(\theta + 90^\circ) = -\sin\theta$, $\sin(\theta + 90^\circ) = \cos\theta$, $\sin(\theta - 90^\circ) = -\cos\theta$ (SymPy ile doğrulandı).

Karmaşık sinyalin önemi şuradan gelir: Fourier dönüşümüne gerçel bir sinyal verdiğimizde çıkış karmaşık olur. Bu yüzden `real` ve `imag` çok kullanılır.

Hoca sinyali koda yazarken frekansı görünür kılmayı tercih etti. $100\pi t$ yerine $2\pi \cdot 50\, t$ yazdı. Sonuç değişmez, ama frekans bilgisi doğrudan görünür. Python'da j, `2j` biçiminde yazılır.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-0.02, 0.05, 1000)
x = np.exp(2j*np.pi*50*t)

plt.subplot(2,1,1)
plt.plot(t, np.real(x))
plt.title(r'Real part of $x(t)=e^{j100\pi t}$')

plt.subplot(2,1,2)
plt.plot(t, np.imag(x))
plt.title(r'Imaginary part of $x(t)=e^{j100\pi t}$')
plt.show()
```

Kod çalıştırıldı, iki grafik ve LaTeX biçimindeki başlıklar çıktı. `$...$` işaretleri arasına yazılan LaTeX kodu başlıkta denklem olarak görünür. Hoca bunu başka derslerde de öğrenmenin faydalı olacağını söyledi.

> **Ders sonrası not:** Hoca baştaki `r` için önce renk olabileceğini, sonra "buradaki şeyin bir denklem olacağını gösteriyor" diye tahmin etti. `r`, Python'da ham metin (raw string) demektir. Ters bölü işaretini kaçış karakteri saymaz: `r'\pi'` üç karakterlidir (denendi). Denklem görünümünü `$` işaretleri sağlar.

Derste önce az örnekle çizim yapıldı ve grafikler bozuk göründü. Örnek sayısı 1000 yapılınca beklenen biçim geldi. Başlıkların üst üste binmesinin sebebini hoca pencere alanının küçük olmasına bağladı.

Soru (hoca): Hangi grafik kosinüs, hangisi sinüs?

Cevap: Üstteki kosinüstür, çünkü $\cos 0 = 1$ ve gerçel kısmın kosinüs olduğunu baştan bulmuştuk. Alttaki sanal kısım sinüstür, $\sin 0 = 0$. Grafikler arasında 90° faz farkı görülür.

12. Sonuçları Grafikle Raporlamak

Çizimlerde sürekli zamanlı bir sistem için `plot`, ayrık gösterim için `stem` kullanılır. Şimdiye kadarkiler zaman gösterimiydi. $x(t)$'ye bir dönüşüm uygulayıp $X(\omega)$ veya $X(f)$ bulduğumuzda çizim aynıdır, yalnız yatay eksen artık frekans eksenidir. Hoca, dijital sinyal işleme dersinde zaman ve frekans eksenini birlikte işleyen yöntemlerin görüleceğini, makine öğrenmesi gibi alanlarda bunların çok kullanıldığını söyledi.

> **Hocanın vurgusu:** Raporlamada eksikler oluyor. Grafikleri özenle çizin. Eksen etiketi, başlık, lejant ve `subplot` kullanın. Fourier dönüşümü veya konvolüsyon gibi bir algoritmayı kendiniz yazdığınızda sonuçları da böyle çizdireceksiniz.

13. Yedek Kaynaktan Örnekler ve Meksika Şapkası

Hoca yeni bölüme geçmeden önce yedek kitap olan M. J. Roberts'ı gösterdi. Kitapta sinyaller güzelce sınıflandırılır. Birim basamak, birim rampa gibi fonksiyonlar ayrıntılı anlatılır. Ayrıca sinyaller üzerinde işlemlerle ilgili ek örnekler vardır: öteleme ve ölçekleme (shifting and scaling), fonksiyonların birleşimi (toplam, çarpım, bölüm), türev, integral, tek ve çift kısım, periyodik sinyaller, enerji ve güç.

Hoca kitaptaki birleşik işlemlere de değindi. Birden fazla dönüşüm varsa önce genlik ölçeklemesi, sonra t/a dönüşümü ve öteleme yapılabilir. Kitapta sıra değişince sonucun değiştiği gösterilir. Hangi sonucu elde edeceksek t üzerindeki dönüşümü ona göre ayarlamamız gerekir.

Hoca kitaptaki bir grafiğin adını sordu ve bunun bize bir şey çağrıştırması gerektiğini söyledi. Bu, çan eğrisinden türetilen bir sinyaldir. Adı Meksika şapkasıdır (Mexican hat), Meksikalıların şapkasına benzer. Hoca literatürde bunun Gauss eğrisinin ikinci dereceden türevi olarak geçtiğini söyledi.

> **Ders sonrası not:** Şekil, Gauss eğrisinin ikinci türevinin negatifidir. $g(t) = e^{-t^2/2}$ için $g''(t) = (t^2 - 1)g(t)$ bulunur (SymPy ile hesaplandı). Bu ifade $t = 0$'da -1'dir. Şapka biçimi, yani merkezde tepe, $-g''(t) = (1 - t^2)e^{-t^2/2}$ ile elde edilir.

14. Sistem Kavramı ve Blok Diyagramları

Aradan sonra hoca sistemlere geçti. Şimdiye kadar sinyalleri, özelliklerini ve üzerlerindeki temel işlemleri konuştuk. Sistemleri de sinyaller gibi hem ayrık hem sürekli zamanda ele alacağız. Hocaya göre sistem aslında çok daha geniş bir kavramdır, ama burada işin sinyal boyutuna bakıyoruz.

Sistem, bir giriş sinyalini alıp çıkış sinyaline dönüştüren bir dönüşüm işlemidir. Aslında bir fonksiyondur, giriş sinyallerini çıkış sinyallerine eşler (map). Biz **tek girişli, tek çıkışlı** (single input, single output) sistemlerle ilgileneceğiz. Bir $x(t)$ olacak, ona karşılık bir $y(t)$ olacak.

Matematikteki $y = f(x)$ gibi burada $y = S\{x\}$ veya $y = Sx$ yazarız. Bu, x'i giriş olarak alıp y'yi çıkış olarak veren S sistemi demektir. Sx yazımı çarpmayla karıştırılabilir, ama çarpma değildir.

Sistemleri çoğunlukla **blok diyagramlarıyla** gösteririz. Bu diyagramlar haberleşme ve kontrol sistemleri gibi alanlarda modellemeyi, programlamayı ve indirgeme gibi işlemleri kolaylaştırır. Oklu çizgiler kablo veya tel değildir, sinyalleri gösterir. Kutu ise sistemin kendisidir. Bazı işlemler için integral gibi özel semboller kullanılır.

15. Temel Sistemler

15.1 Ölçekleyici

Sinyallerde ölçekleme yaptığımız gibi sistem de bir sinyali ölçekleyebilir: $y(t) = Ax(t)$. Dikkat edelim, bu **genlik üzerinde** bir ölçeklemedir ve bir sistemin görevidir. Örneğin bir uygulamadaki veya teypteki ses açma düğmesi bir alt sistemdir. Görevi genliği yükseltmektir.

- $|A| > 1$: Yükselteç, güçlendirici (amplifier).
- $0 < |A| < 1$: Zayıflatıcı (attenuator). Hoca "attenuate" etmenin söndürmek, yok etmeye yaklaştırmak anlamına geldiğini söyledi.
- $A < 0$: Evirici (inverter).

A parametresine kazanç (gain) veya ölçek faktörü denir. Blok diyagramında üçgen veya daire ile çizilir, içine A yazılır.

15.2 Türev Alıcı

Sistem türev alıyorsa türev operatörü kullanılır: $y(t) = \frac{d}{dt}x(t)$. Türev, bir bağımsız değişkene göre değişimi gösterir. Girişteki değişimi çıkışta gösterir.

Soru (hoca): Girişte hız varsa çıkışta ne elde ederiz?

Cevap: İvme (acceleration). Hoca bunu "hızın hızı" diye de özetledi. Hızdaki değişim, yani hızlanma ivmedir.

15.3 İntegratör

Sistem integral alıyorsa integral sembolü kullanılır:

$$y(t) = \int_{-\infty}^{t} x(\tau)\, d\tau$$

Hoca burada τ gösterimini açıkladı. Karmaşık sayılarda i akımla karışmasın diye j kullanıyorduk. Benzer şekilde zaman değişkeni bazen τ ile gösterilir. Ayrıntısı konvolüsyonda gelecektir.

15.4 Zamanda Öteleme

Sistem sinyali zaman ekseninde kaydırıyorsa buna zamanda öteleme sistemi (time shift) denir. Gecikme $y(t) = x(t - t_0)$, öne alma veya öngörü (predictor, advance) $y(t) = x(t + t_0)$ biçimindedir. Bunun için özel bir sembol yoktur, kutunun içine öteleme miktarı yazılır.

15.5 Konvolüsyon

Hoca bunu bizim için önemli diye işaretledi. Bir sisteme giriş uygulandığında elde edilecek çıkış, konvolüsyon işlemiyle bulunur:

$$y(t) = x(t) * h(t) = \int_{-\infty}^{\infty} x(\tau)\, h(t - \tau)\, d\tau$$

Hoca önce sistemi $s(t)$ ile yazdı, sonra slaytta daha çok kullanıldığı için $h(t)$ dedi. Çarpımlar τ boyunca toplanır, yani integre edilir. Burada eksen değiştiririz: Değişken t'den τ'ya geçer. Hoca $h(\tau)$ ile $x(t - \tau)$ çarpımı olarak da anlattı.

> **Hocanın vurgusu:** Sinyal ve sistem dersinde yıldız işareti çarpma değil, konvolüsyondur. Açılımı yukarıdaki integraldir. Bunu kesinlikle karıştırmayın.

Fourier dönüşümünün de konvolüsyonla ilgili olduğunu, ileride çok duyacağımızı söyledi. Daha önce gösterdiği görüntü işleme örneğini hatırlattı: Bir yüz üzerinde gözün bulunması. Orada göz, görüntünün bütün değerleriyle konvole edilir. Bu, bir anlamda korelasyondur. Korelasyon iki şey arasındaki ilişkidir. İki şey birbirine çok benziyorsa korelasyon yüksektir. Sonuçtaki beyaz noktalar bu yüksek değerleri gösterir.

Ders sonrası not: "Girişin çıkışı konvolüsyonla bulunur" ifadesi yalnız doğrusal ve zamanla değişmez (LTI) sistemler için geçerlidir. Ayrıca iki biçim aynıdır: $x*h = h*x$ (Python ile denendi). Göz arama örneği teknik olarak korelasyondur. Konvolüsyonda kayan sinyal ters çevrilir, korelasyonda çevrilmez.

15.6 Toplayıcı, Fark ve Çarpıcı

x_1 ve x_2 işaretleri toplanıp y elde ediliyorsa bu toplama sistemidir. Fark işlemi ve iki sinyalin çarpımı için de benzer bloklar kullanılır.

16. Sistem Bağlantıları

Sistemler bir arada kullanılabilir (interconnection):

- **Kaskat (cascade, seri):** Giriş bir sisteme girer, çıkışı başka bir sisteme girer. $y = G(F(x))$.
- **Paralel:** x'i hem F hem G alır. Çıkışları toplanır. $y = F(x) + G(x)$.
- **Geri besleme (feedback):** Bir sistemden beklenen sonuç alınamadıysa bu sonuç geriye yansıtılıp sistem daha iyi hâle getirilir. F, girişi ve geri gelen sinyali birlikte alır. y, G sisteminden geçip girişten çıkarılır: $x - G(y)$.

Geri beslemede çıkarma yapılmasının sebebini hoca sosyal hayattan bir olguyla anlattı: Yanlış bir şey yapıyorsan onun tersine hareket edersen doğruyu bulursun. Sebepleri değiştirmeden sonuç değişmez. Bizi yanlışa götüren şeyin tersine doğru çıkararak istediğimiz sonuca ulaşmaya çalışırız.

Bu semboller yalnız gösterim değildir. Sistemlerin denklemlerini de yazacağız. Giriş verilince çıkışın ne olacağını bulacak ve sistemin özelliklerini analiz edeceğiz.

17. İnsan Öğrenmesi Modeli ve PID Denetleyici

17.1 İntegratör ve Geri Besleme

Hoca bir integratör ile geri beslemeyi birleştirince bunun insanın eğitimiyle ilgili bir model olduğunu söyledi ve yorum istedi.

Soru (hoca): Bu şekil günlük hayatımızla nasıl ilişkilidir?

Cevap (öğrenci): Gün içinde bir şeyler yaşıyoruz. Bunların toplamı y oluyor. Sonra geri döndürüyoruz. Yanlışımızı belli bir sayıyla çarpıp azaltıyor ve ona göre tekrar yaşıyoruz.

Cevap (hoca): Evet, ama bir eksik var. Yaşadıklarımızı hata olarak görüyoruz ve hatalarımızı topluyoruz. Bu toplam, tecrübe dediğimiz şeydir. Hoca bunu aslında doğru bir tanım saydı. Çıktılarımız olan y'leri belli bir ölçekten, belli bir filtreden geçirip sürekli topluyoruz. Bu toplamaya dış etkenler de, yani yaşadığımız diğer girişler de katılır.

Eksik olan parça türevdir. Bugünkü tecrübemi bir gün önceki tecrübemle karşılaştırırsam daha güzel bir model olur. Hoca bunu türevle birlikte düşünmemizi söyledi. Kontrol sistemlerinde bu yapıya **PID denetleyici** denir.

17.2 Aç-Kapa Denetleyici

Hoca PID'nin neden gerektiğini oda sıcaklığı örneğiyle anlattı. Odanın sıcaklığını 20 ile 30 derece arasında tutmak istiyoruz. Isıtıcı başlangıçta sıfırdan çalışır.

1. Isıtıcı açıktır, sıcaklık 30'a kadar yükselir.
2. 30'da ısıtıcı kapanır. Kapanınca sistem sıfırdaymış gibi davranır, sıcaklık kendiliğinden düşer.
3. Sıcaklık 20'yi görünce ısıtıcı tekrar çalışır.
4. 30'a gidince yine kapanır.

Buna **aç-kapa denetleyici** (on-off controller) denir. Yapılan bu harekete de **salınım** (oscillation) denir. Hoca bunun bir bilgisayarcıya mantıklı görünebileceğini, ama sistemi tasarlayan biri için sürekli aç-kapa yapmanın iyi olmadığını söyledi.

Bunun yerine sıcaklığın düştüğünü fark edip, bir sonraki adımda bu kadar düşmesine izin vermeden daha hızlı davranırsak, öğrendiğimiz tecrübeyi kullanmış oluruz. İdeal davranış şudur: Bir referansımız vardır. Ona ulaşana kadar gideriz. Önce büyük hata yaparız, hatalar yavaş yavaş azalır, sonunda referansa otururuz. İntegral ve türev sistemleri buna yakın bir model üretir.

17.3 Çamaşır Makinesi Örneği

Hoca bunu çamaşır makinesinin devir sayısıyla örneklendirdi. 1000 devirli bir makine bir anda 1000 devire çıkmaz. Elektrik enerjisi dönme hareketine dönüşür ve belli bir deviri tutturacak voltaj ve akım değerleri vardır. Hoca işin mekanik tarafını çok bilmediğini belirtti. Sistem söz konusu devire yaklaşarak gider ve orada kararlı kalması gerekir.

PID, oransal, integral ve türev (proportional, integral, derivative) sözcüklerinden gelir. Şekilde oransal kısım, yani katsayı, ve integral vardır. Yalnız türev yoktur. Türev de eklenince bu sistemler daha kararlı, hocanın deyişiyle daha akıllı ve uslu çalışır.

İnsan için de benzer bir sosyal boyut vardır. Kişi hatalarını fark edip toplar, geçmişten ders çıkarır, bunları tecrübeye dönüştürür ve bu tecrübeyi iyiye doğru geliştirirse mükemmele doğru bir gelişim olur. Hoca türevin bir anlamda şimdiki hâlimizden öncekini çıkarmak olduğunu, geri beslemeyle birlikte düşünülebileceğini ekledi.

> **Ek bilgi (derste anlatılmadı):** PID denetleyicinin çıkışı büyük olasılıkla hata $e(t)$ üzerinden şöyle yazılır: $u(t) = K_p\, e(t) + K_i \int e(\tau)\, d\tau + K_d\, \frac{de(t)}{dt}$. Burada oransal terim bugünkü hataya, integral terimi birikmiş hataya, türev terimi hatanın değişimine bakar. Dersteki “hataların toplamı” ve “bugünü dünle karşılaştırma” benzetmeleri büyük olasılıkla bu iki terime karşılık gelir.

18. Doğrusallık

Sistem denen olgu çok farklı biçimlerde karşımıza çıkar. Biz bilgisayar mühendisleri olarak belli sistemlerle ilgileniyoruz. Her türlü sistem ilgi alanımıza girmez. Bu alanı daraltmak için kullanacağımız özellikler doğrusallık ve zamanla değişmezliktir. İkisini sağlayan sistemlere **doğrusal ve zamanla değişmez** (linear time invariant, LTI) sistemler denir. En çok bu sistemler üzerinde çalışacağız.

18.1 Tanım

Bir sistem **homojenlik** ve **süperpozisyon** özelliklerini sağlıyorsa doğrusaldır.

- **Homojenlik, çarpımsallık (multiplicativity):** $S\{ax\} = aS\{x\}$. Ölçek faktörü sistemin öncesinde veya sonrasında uygulansa da sonuç aynıdır.
- **Süperpozisyon, toplamsallık (additivity):** $S\{x_1 + x_2\} = S\{x_1\} + S\{x_2\}$. Girişleri toplayıp sisteme vermek ile ayrı ayrı verip sonuçları toplamak aynıdır.

İkisi tek testte birleştirilebilir: $S\{ax_1 + bx_2\} = aS\{x_1\} + bS\{x_2\}$. Hoca bu birleşik testin kafa karıştırabileceğini, adım adım önce birine, sonra diğerine bakmanın en güvenli yol olduğunu söyledi.

Blok diyagramıyla düşünürsek: x'i a ile çarpıp sisteme vermek ile sistemden çıkanı a ile çarpmak aynı olmalıdır. $x_1 + x_2$'yi sisteme vermek ile ayrı ayrı verip sonuçları toplamak da aynı olmalıdır.

> **Hocanın vurgusu:** Bir sistem toplamsallığı sağlayıp çarpımsallığı sağlamayabilir, ya da tersi. Doğrusal olması için ikisini de sağlaması gerekir. Biri bile sağlanmazsa doğrusal değildir.

18.2 Örnek: $f(t) = 2t$

Hoca testi $f(t) = 2t$ örneğiyle gösterdi.

- **Homojenlik:** $f(at) = 2at$, $a\, f(t) = a \cdot 2t = 2at$. Eşittir.
- **Toplamsallık:** $f(t + a) = 2(t + a) = 2t + 2a$, $f(t) + f(a) = 2t + 2a$. Eşittir.

İki özellik de sağlandığı için doğrusal dedi.

18.3 Örnek: $2t + 5$

Hoca bu basit örneği, öğrenciler fonksiyonlar ve analitik geometriyle çok karıştırdığı için yaptı. $y = 2x+5$ bir fonksiyon olarak doğrudur. Eğimi 2 olan, ekseni 5'te kesen bir doğru çizilir. Ama bu doğrusal bir sistem değildir.

- **Çarpımsallık:** $a\,x(t) = a(2t + 5) = 2at + 5a$. $x(at) = 2at + 5$. Eşit değildir.
- **Toplamsallık:** $x(t+a) = 2(t+a)+5 = 2t+2a+5$. $x(t)+x(a) = (2t+5)+(2a+5) = 2t + 2a + 10$. Eşit değildir, çünkü $5 \neq 10$.

İki özellik de sağlanmadığı için sistem doğrusal değildir. Birincinin sağlanmaması zaten yeterlidir.

> **Ders sonrası not:** Derste test t üzerinden yapıldı: $f(at)$ ile $af(t)$ karşılaştırıldı. Bu, girişi değil zamanı ölçekler. Doğru test girişi ölçekler: $S\{a\,x(t)\}$ ile $a\,S\{x(t)\}$ karşılaştırılır. Sonuçlar değişmez. $y(t) = 2x(t)$ doğrusaldır. $y(t) = 2x(t) + 5$ için $S\{ax\} = 2ax+5$, $aS\{x\} = 2ax+5a$ çıkar ve sistem doğrusal değildir. Toplamsallıkta da 5 birim fark kalır. Rastgele girişlerle Python'da iki testi de denendi, sonuç aynıdır.

Alıştırma: Hoca $y(t) = x^2(t)+5$ sisteminin doğrusallığını kendi başımıza test etmemizi istedi.

> **Ders sonrası not:** Sonuç kontrol için: $S\{ax\} = a^2x^2+5$, $aS\{x\} = ax^2+5a$ olur ve eşit değildir (SymPy ile hesaplandı). Toplamsallık da rastgele girişlerle sağlanmadı. Sistem doğrusal değildir.

19. Doğrusal ve Doğrusal Olmayan Sistem Örnekleri

Doğrusal sistem örnekleri: ölçekleyici, türev alıcı (differentiator), integratör, hareketli ortalama (running average), zamanda öteleme, konvolüsyon, modülasyon. Türev alan veya integral alan bir devre tasarlanabilir, aynı işlemler sayısal tarafta da yapılabilir.

Hareketli ortalama: Bir sinyal üzerinde bir pencere seçilir. Örneğin görüntüde 3×3 boyutunda bir pencere. Penceredeki değerlerin ortalaması alınıp ortadakine yazılır, sonra pencere kaydırılır. Bu ortalama işlemi doğrusal bir sistemdir.

Doğrusal olmayan örnekler: kare alıcı, bazı çarpıcılar, karşılaştırıcılar, nicemleyiciler (quantizer), sayısal uyarlamalı (adaptive) filtreler. Hoca özellikle uyarlamalı filtrelerin doğrusal olmadığını vurguladı. Bu filtreler doğrusal sistemler cinsinden tasarlanabilir. Doğrusal olmayan bir sinyal parçalara bölünüp doğrusal sistemlerle işlenebilir.

Kare alıcı $y(t) = x^2(t)$ doğrusal değildir. Hoca bunu ispatlamanın yukarıdaki iki koşulu uygulamak olduğunu söyledi.

19.1 Modülatör

Hoca modülatörü çok yaygın bir doğrusal işlem olarak gösterdi. Modülatör aslında bir dönüştürücüdür. Bir $x(t)$ işaretini belli bir frekans aralığına çeker:

$$y(t) = x(t)\cos(\omega_c t)$$

Örnek olarak insan sesini verdi. İnsan sesi, hocanın ifadesiyle galiba 10 Hz'den 10 kHz'e kadar bir aralığı içerir. Telefonla konuşurken kablolar, yani iletim hatları, bu aralıkta çalışmaz. Sinyali başka bir aralığa çevirmek için modüle etmek gerekir. İşlem aslında bir çarpmadır. Kosinüs işaretine **taşıyıcı sinyal** (carrier) denir. Hoca haberleşmecilerin bu konuda çok daha derin bilgiye sahip olduğunu söyledi.

> **Ders sonrası not:** Hoca "kablolar bu aralıkta çalışmaz, daha küçük aralığa çevirmek gerekir" dedi. $x(t)\cos(\omega_c t)$ çarpımı sinyalin frekans içeriğini ω_c çevresine taşır. Yani amaç büyük olasılıkla "daha küçük aralık" değil, sinyali kanalın çalıştığı banda taşımaktır. Modülatör doğrusaldır, ama zamanla değişendir: Python'da bir test sinyalinde $x(t - t_0)\cos(\omega_c t)$ ile $x(t - t_0)\cos(\omega_c(t - t_0))$ farklı çıktı.

20. Bellekli ve Belleksiz Sistemler

Sistemler bellek kullanıp kullanmamasına göre ikiye ayrılır.

- **Belleksiz:** Çıkış yalnız şu anki girişe bağlıdır. $y(t)$, doğrudan $x(t)$'ye bağlıdır.
- **Bellekli:** Çıkış başka değerlere de ihtiyaç duyar. Bu değerler geçmişten de gelecekten de olabilir.

Örneğin $y(t) = x(t) + x(t-1)$ şimdiki girişle birlikte bir önceki ana da ihtiyaç duyar, bu yüzden ayrıca bir bellek gerekir. $y(t) = x(t) + x(t+1)$ de belleklidir. Bu ikincisi teoride mümkündür, ama uygulamada bir saniye sonrasının değerini almak mümkün değildir. Bu tür sistemler gecikmeli olarak kullanılır: t yerine $t-1$, $t+1$ yerine t alınarak.

- **Belleksiz fiziksel örnekler:** İdeal yükselteç (amfi), dişli (gear), kaldıraç (lever). Bunlar anlık çalışır.
- **Bellekli fiziksel örnekler:** Enerji depolayan kondansatör ve bobin, önceki değerleri kullandığı için belleklidir. Mekanik sistemlerde yay ve hareketli kütle de belleklidir.

21. Nedensellik

Hoca önce bir kelime karışıklığına dikkat çekti. *Causal* kelimesi *cause* kökünden gelir, neden-sonuç anlatır. *Casual* ise günlük kıyafetlerde kullanılan kelimedir. Causal sistem, nedensel sistem demektir. İkisi karıştırılmamalıdır.

Nedensel sistemin çıkışı yalnız şimdiki ve önceki girişlere bağlıdır. Bir şeyin nedeni olması için öncesinde bir şeyler olması gerekir. Geleceğe yönelik nedensellik olmaz. Hocanın örneği: "Sınıfı geçtim, çünkü çok çalıştım." Önce çalıştım, sonra geçtim. Bir şeyin nedeni önce veya şu an olur. "Çalışırsam sınıfı geçerim" ise bir şarttır, nedensel değildir.

Fiziksel devreler ve mekanik sistemler nedenseldir. Hoca bellek ile nedenselliğin farklı kavramlar olduğunu özellikle belirtti. Bellekli bir sistem, Bölüm 20'deki $x(t+1)$ örneğinde olduğu gibi nedensel olmayabilir.

22. Zamanla Değişmezlik

Değişmezlik (invariant) sözcüğü adından anlaşılır, varyant olsa değişir. Girişte yaptığımız bir gecikme çıkışta da aynı sonucu veriyorsa sisteme **zamanla değişmez** denir. Girişte t_0 kadar bir kayma sağladığımızda çıkışta da aynı kaymayı elde etmeliyiz.

Algoritma şöyledir:

1. Girişe gecikme uygula ve sisteme ver: $y_1(t) = S\{x(t-t_0)\}$.
2. Girişi sisteme ver, çıkışa gecikme uygula: $y_2(t) = y(t-t_0)$.
3. $y_1 = y_2$ ise sistem zamanla değişmezdir, değilse zamanla değişendir.

Hoca örnek olarak $y(t) = 2t$ yazdı. Girişte $t-t_0$ yazınca $2(t-t_0) = 2t-2t_0$ buldu, bunu diğer ifadeyle karşılaştırdı, eşit olmadığını söyleyip sistemin zamanla değiştiği sonucuna vardı.

> **Ders sonrası not:** $y = 2t$ yazımında hangi sistemin kastedildiği kayıtta net değil, doğrusallıktaki gibi t giriş yerine kullanılmış. İki olası sistemin sonucu farklıdır (test sinyaliyle Python'da denendi). $y(t) = 2x(t)$ zamanla değişmezdir: $S\{x(t-t_0)\} = 2x(t-t_0) = y(t-t_0)$. $y(t) = x(2t)$ zamanla değişendir: $S\{x(t-t_0)\} = x(2t-t_0)$, ama $y(t-t_0) = x(2t-2t_0)$. Hocanın vardığı "değişir" sonucu ve bulduğu $2t-2t_0$ ifadesi büyük olasılıkla ikinci sisteme karşılık gelir.
>
> **Hocanın vurgusu:** Doğrusallık ve zamanla değişmezlik bir araya gelince sistemleri çok daraltacağız. Bundan sonra anlatacağımız konular, özellikle konvolüsyon, doğrusal ve zamanla değişmez sistemler üzerine kuruludur.

23. Kararlılık

Günlük dilde "sistem stabil çalışmıyor" deriz. Hoca bunu önce şöyle örneklendirdi: Bir giriş verdim, bir çıkış aldım. Başka zaman aynı girişi verdim, başka bir çıkış aldım.

Burada asıl kastedilen tanım şudur: Sınırlı bir giriş kümesindeki her girişe karşı sınırlı çıkışlar elde ediyorsak, sistem hiçbir zaman sonsuza gitmiyorsa, sistem kararlıdır. Buna **BIBO** (bounded input, bounded output) kararlılığı denir:

$$|x(t)| \leq M_x < \infty \text{ için } |y(t)| \leq M_y < \infty$$

> **Ders sonrası not:** Aynı girişe farklı çıkış vermek BIBO kararlılığının konusu değildir. O örnek büyük olasılıkla günlük dildeki anlamı anlatıyor. BIBO'da yalnız çıkışın sınırlı kalıp kalmadığına bakılır.

Bunu zaman düzleminde test etmek biraz zordur. Çünkü girişlerin sonlu olup olmadığını denetlemek gerekir. Girişler bütün gerçel sayılar olursa küme sonlu olmaz. Frekans düzlemine veya z-düzlemine geçince test daha kolaydır. Sistemin fonksiyonu üzerinden çok daha kolay test edeceğiz.

24. Örnek Sistem: Hız Sabitleyici

Hoca sistem özelliklerinden sonra örnek olarak hız sabitleyiciyi (cruise control) gösterdi. İstenen bir hız vardır. Araba bu hıza gaz pedalına basınca doğrudan ulaşamaz. Sürücü hızı ekranda görür ve referans değerden bir hata üretir. "Gaza çok bastım, ayağımı biraz çekeyim" ya da "az bastım, biraz daha yükleneyim" diye kendini ayarlar. Bu işlem referans değere ulaşana kadar devam eder.

Blok diyagramında x istenen hız, y ölçülen hızdır. Aralarındaki fark K ölçekleme parametresiyle çarpılır ve araç sistemine girer:

$$K(x - y)$$

Negatif hata olursa gaz kesilir. Hız değişimindeki gecikme de hedefin altına düşmeye yol açar. Hoca böyle bir algoritma tasarlarsak istediğimiz hıza gidebileceğimizi söyledi.

> **Ders sonrası not:** Hoca burada "aslında türev alıyoruz" dedi. Şekildeki yapı hatayla orantılı bir denetleyicidir, $K(x - y)$ ifadesinde türev yoktur. PID diliyle bu oransal (P) kısımdır.

25. Terslenebilirlik

Bir sistem, tersiyle ard arda bağlandığında birim işleci (identity operator) veriyorsa terslenebilirdir.

> **Ders sonrası not:** Hoca tanımı "sistemin tersi kendisine eşitse" diye söyledi. Doğrusu: Sistem ile tersinin kaskat bağlantısı girişi aynen geri vermelidir, $S^{-1}\{S\{x\}\} = x$. Birim işleci olan işlem de bu kaskat bağlantıdır.

26. Diferansiyel Denklemlerle Tanımlanan Sistemler

Hoca konuya gündelik bir formülle girdi: Yol, hız çarpı zamandır, $x(t) = v(t) \cdot t$. Burada aslında türevli bir durum vardır.

> **Ders sonrası not:** $x = vt$ yalnız sabit hızda geçerlidir. Genel ilişki $v(t) = dx(t)/dt$'dir.

Bazı sistemler diferansiyel denklemlerle ifade edilir. Örneğin $y(t)$ girişin ikinci veya beşinci türeviyle çalışabilir. Ya da $y(t)$'nin türeviyle kendisi toplanır, girişin üçüncü türevi alınır. Genel biçim şöyledir:

$$\sum_k a_k \frac{d^k y(t)}{dt^k} = \sum_k b_k \frac{d^k x(t)}{dt^k}$$

- $\frac{d^k}{dt^k}$ türev operatörleridir.

- a_k, b_k sabit katsayılardır, örneğin 3, 5, 7 gibi sayılar.
- $y(0), y'(0), \ldots$ değerleri **başlangıç koşullarıdır**.
- En yüksek türevin derecesi sistemin **mertebesidir**.

Hoca bu denklemleri ileride dönüşümler yoluyla daha rahat çözeceğimizi söyledi. Sabit katsayılı sistemler, başlangıç koşullarının tamamı sıfırsa doğrusaldır.

26.1 RLC Devresi

Hoca devre teorisinden RLC devresini hatırlattı. Devre ya fazörlerle (vektörel olarak) ya da türev ve integral alarak çözülür. Türev-integral yolunda aslında ikinci mertebeden bir diferansiyel denklem çözülür. Kirchhoff gerilim yasasından:

$$LC\frac{d^2y(t)}{dt^2} + RC\frac{dy(t)}{dt} + y(t) = x(t)$$

İkinci türevin katsayısı endüktans ile kapasitansın, birinci türevin katsayısı direnç ile kapasitansın çarpımıdır. Üçüncü terim sistemin o anki değeridir. Toplam girişe eşittir. Başlangıç koşulları sıfırsa bu doğrusal bir sistemdir.

> **Ders sonrası not:** Altyazıda y'nin ne olduğu söylenmiyor. Bu denklem, y kondansatör gerilimi alınırsa çıkar: $Li' + Ri + y = x$ ve $i = Cy'$ yerine yazılınca aynı ifade elde edilir (SymPy ile hesaplandı).

26.2 Yay-Kütle Sistemi

Bir başka örnek mekanik sistemdir: Bir yay, bir kütle ve yayda oluşan gerginlik. Hoca bir makine mühendisinin bu sistemin diferansiyel denklemini yazabileceğini, kendisinin bu sistemlerin davranışına o kadar hâkim olmadığını söyledi. Bu tarz sistemler de bu biçimde bir diferansiyel denklem oluşturur.

27. Fark Denklemleriyle Tanımlanan Ayrık Sistemler

Hoca asıl ilgileneceğimiz sistemlerin bunlar olduğunu söyledi. Sistem sürekli zamana göre değil de $n = 0, 1, 2, 3, \ldots$ gibi bir değişkene göre değişiyorsa ayrık zamanlı sistemdir. $x[n]$ girişi, $y[n]$ çıkışı gösterir.

Sürekli zamandaki diferansiyel denklemin (differential equation) ayrık zamandaki karşılığı **fark denklemidir** (difference equation). Hoca bunların özyinelemeli (recursive) algoritmalarda da anlatıldığını belirtti.

Slayttaki örnek ayrık zamanlı sistemde $y[n]$, bir önceki anın değerleriyle hesaplanır. $y[n-1]$ belli bir miktar ölçeklenir ve girişin bir önceki anıyla birleştirilir:

$$y[n] = a\,y[n-1] + x[n-1]$$

Bu, gecikme biçiminde çalışan bir ayrık zamanlı sistemdir.

Dersin Toparlaması

Ders iki ana parçadan oluştu. Birinci parçada sinyaller kapandı. Quiz soruları üzerinden örnekleme, nicemleme ve veri boyutu hesabı tekrar edildi. $x(-2t+6)$ örneğiyle öteleme, ölçekleme ve terslemenin her zaman t üzerinde yapıldığı gösterildi. Dürtünün örnekleme operatörü olduğu görüldü. Ardından MATLAB ve Python ile sinyal çizimi ele alındı: `plot`, `stem`, `hold on`, `figure`, `subplot`, eksen etiketleri, `real`, `imag`. Bilgisayardaki her sinyalin ayrık olduğu özellikle vurgulandı.

İkinci parçada sistemlere geçildi. Sistem, giriş sinyalini çıkış sinyaline eşleyen bir dönüşümdür. Temel sistemler, bağlantı biçimleri, PID denetleyici fikri ve sistem özellikleri görüldü: doğrusallık, bellek, nedensellik, zamanla değişmezlik, BIBO kararlılığı ve terslenebilirlik. Hoca

bir sonraki derste sistem özellikleriyle ilgili birkaç örnek daha çözeceğini, sonra sistemler ve sinyaller üzerindeki işlemlere geçeceğini söyledi. Doğrusal ve zamanla değişmez sistemler, sıradaki konu olan konvolüsyonun temelidir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.